

#JCON2025
www.jcon.one

Himalayan Peaks

of Testing Data Pipelines

Kseniia Tomak, HelloFresh

Pasha Finkelshteyn, BellSoft



JAVAPRO

whoami

whoami

- Pasha Finkelshteyn

whoami

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft

whoami

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly ☕ and 

whoami

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly ☕ and 🚩
- And 🌿

whoami

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly ☕ and 🚩
- And 🌿
- 🐦 asm0di0

whoami

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly ☕ and 🚩
- And 🍃
- 🐦 asm0di0
- 📧 @asm0dey@fosstodon.org

whoami

whoami

- Kseniia Tomak

whoami

- Kseniia Tomak
- Data Engineering Manager at HelloFresh

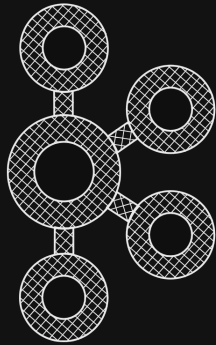
whoami

- Kseniia Tomak
- Data Engineering Manager at HelloFresh
- $\approx$ 14 years of Engineering experience. Mostly Data engineering

whoami

- Kseniia Tomak
- Data Engineering Manager at HelloFresh
- $\approx$ 14 years of Engineering experience. Mostly Data engineering
-  ksenia-tomak

Data processing



topic: actions

modify data →

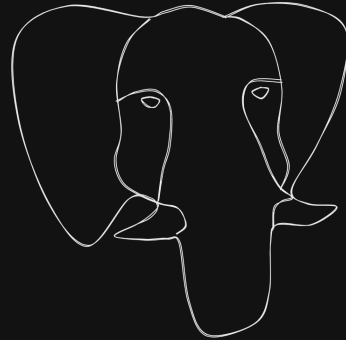
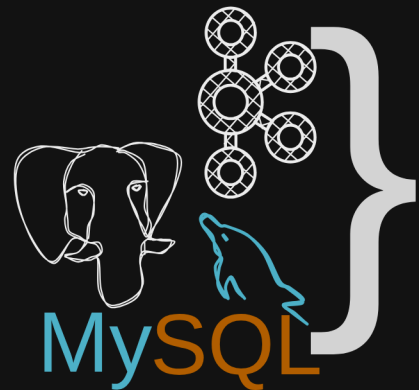


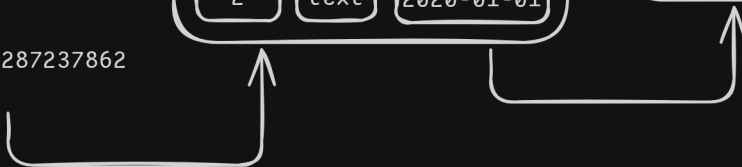
table: actions

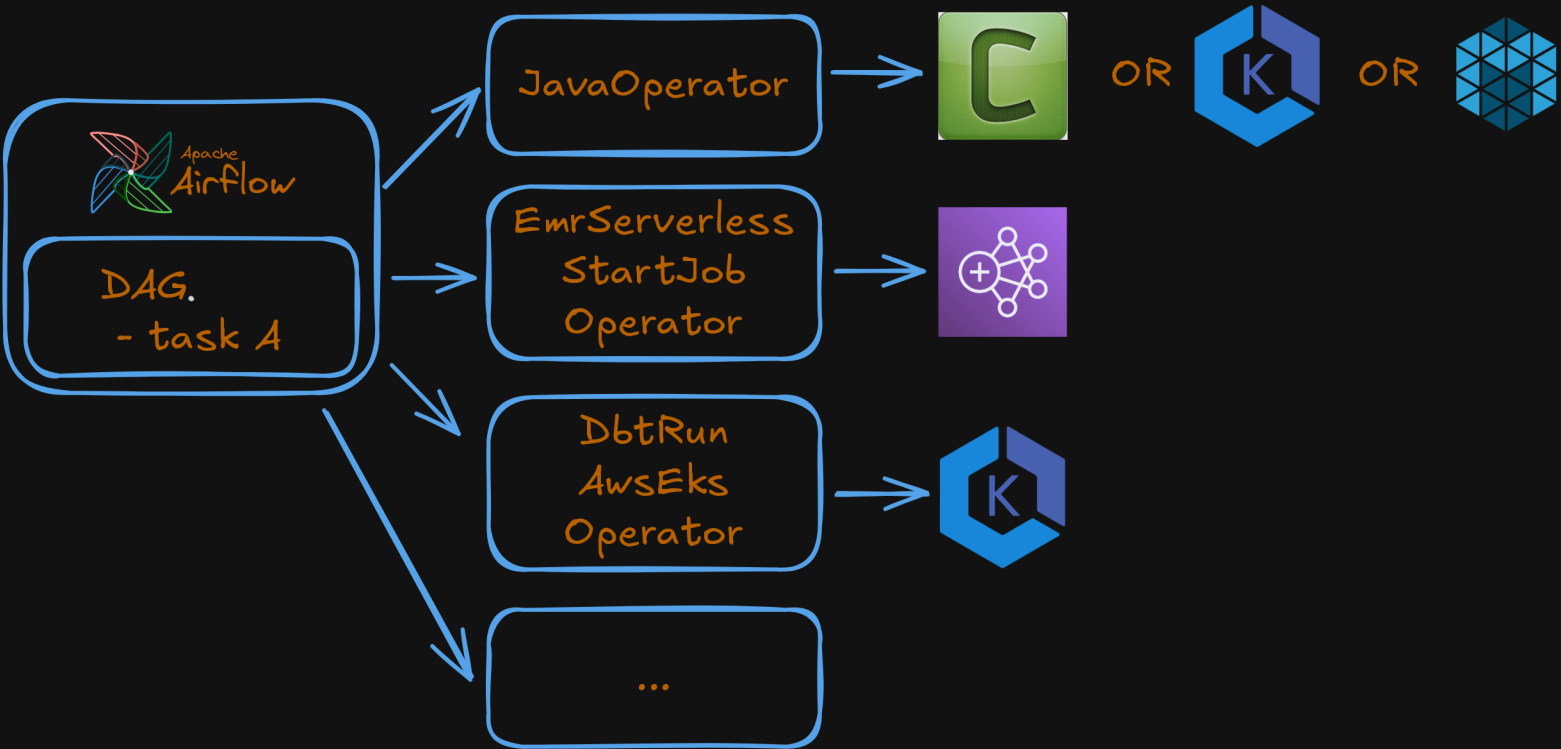
Data Lake?



bronze

```
{  
  "data": {  
    "pk": 2,  
    "aa": "text"  
  },  
  "op": "c",  
  "ts_ms": 1287237862  
}
```





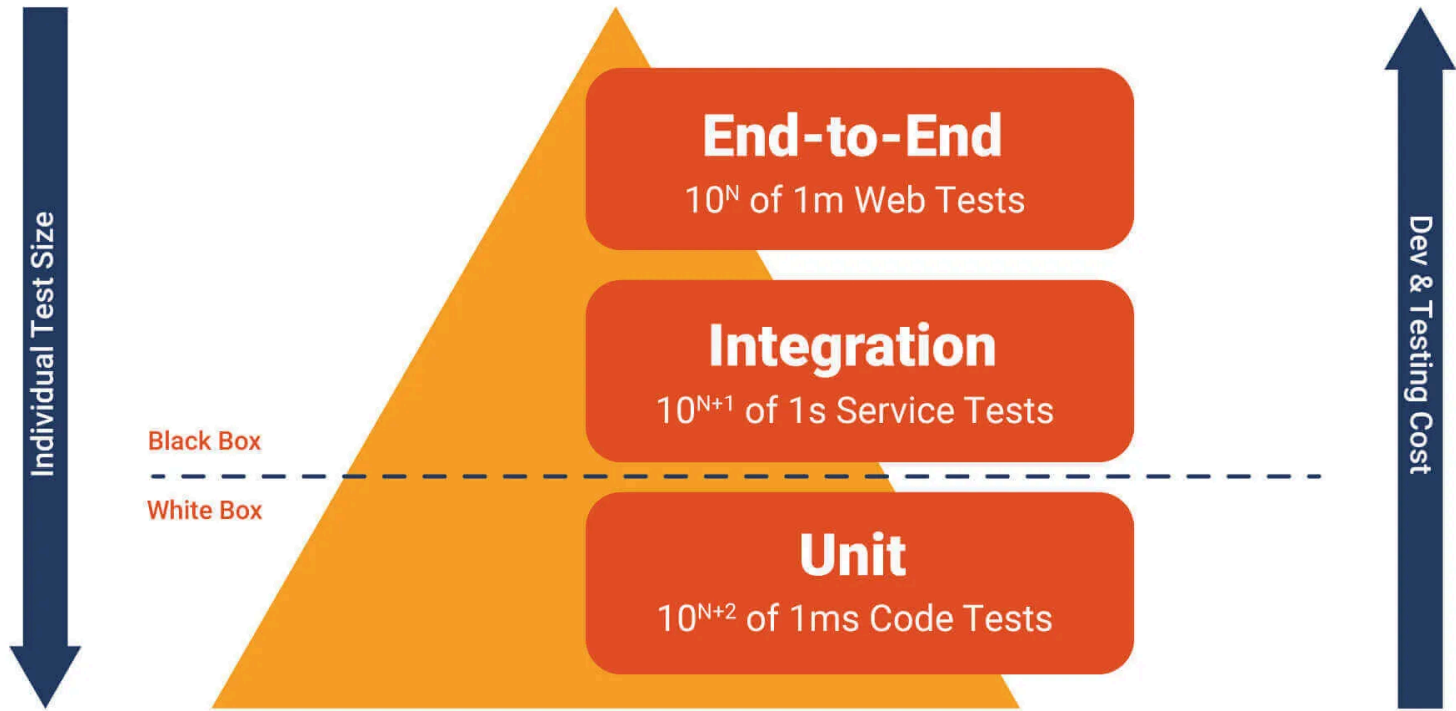
Who needs pipelines

- Data Scientists
- Data Analytics
- Data Engineers
- POs
- any data-driven person

**It has to be
tested**

Pyramid of testing?

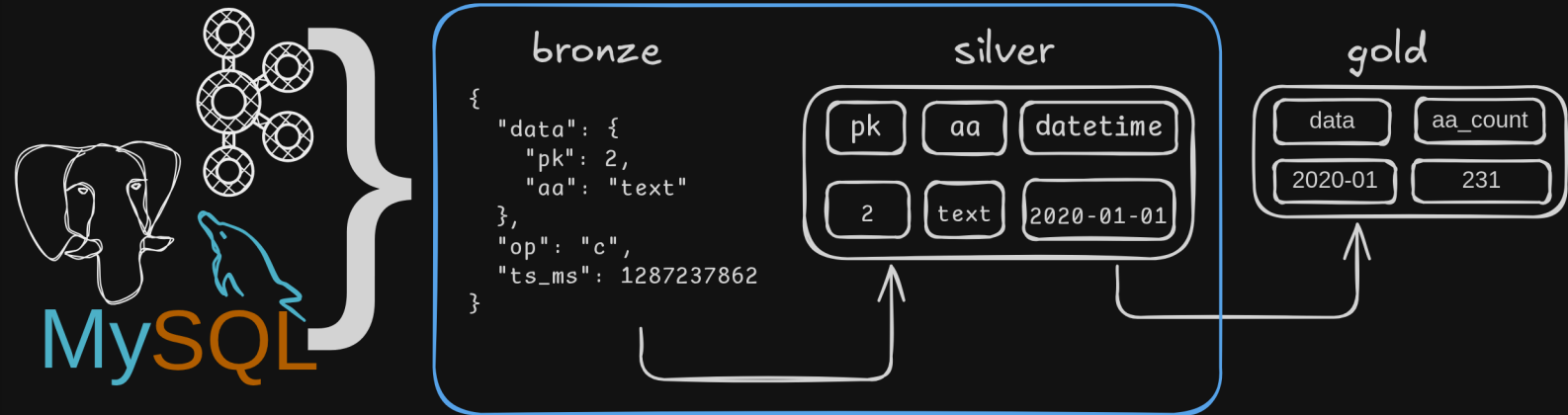
Pyramid of testing?



Pyramid of testing. Unit.



Bronze→Silver pipeline



Typical pipeline

```
1  StructType schema = new StructType(new StructField[]{
2      new StructField("pk", DataTypes.LongType, false, Metadata.empty()),
3      new StructField("aa", new DataTypes.StringType(), false, Metadata.empty())
4  })
5
6  spark.read
7      .schema(schema)
8      .csv(/* path */)
9      .map(/* mapper */)
10     .show() // terminal operation
```



Typical pipeline

```
1  StructType schema = new StructType(new StructField[]{
2      new StructField("pk", DataTypes.LongType, false, Metadata.empty()),
3      new StructField("aa", new DataTypes.StringType(), false, Metadata.empty())
4  })
5
6  spark.read
7      .schema(schema)
8      .csv(/* path */)
9      .map(/* mapper */)
10     .show() // terminal operation
```



Typical pipeline

```
1  StructType schema = new StructType(new StructField[]{
2      new StructField("pk", DataTypes.LongType, false, Metadata.empty()),
3      new StructField("aa", new DataTypes.StringType(), false, Metadata.empty())
4  })
5
6  spark.read
7      .schema(schema)
8      .csv(/* path */)
9      .map(/* mapper */)
10     .show() // terminal operation
```



Typical pipeline

```
1 StructType schema = new StructType(new StructField[]{
2     new StructField("pk", DataTypes.LongType, false, Metadata.empty()),
3     new StructField("aa", new DataTypes.StringType(), false, Metadata.empty())
4 });
5
6 spark.read
7     .schema(schema)
8     .csv(/* path */)
9     .map(/* mapper */)
10    .show() // terminal operation
```



Typical pipeline

```
1  StructType schema = new StructType(new StructField[]{
2      new StructField("pk", DataTypes.LongType, false, Metadata.empty()),
3      new StructField("aa", new DataTypes.StringType(), false, Metadata.empty())
4  })
5
6  spark.read
7      .schema(schema)
8      .csv(/* path */)
9      .map(/* mapper */)
10     .show() // terminal operation
```



Unit testing of pipeline

What may we test here?

A pipeline should transform data correctly!

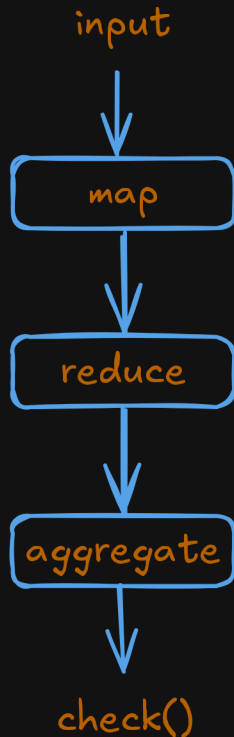
Correctness is a business term

Let's paste fakes!

Fake input data

Reference data at the end of the pipeline

Single function



Tools

[holdenk/spark-testing-base](#) ← Tools to run tests

[MrPowers/spark-daria](#) ← tools to easily create test data

When Should You Run It?

On CI/CD

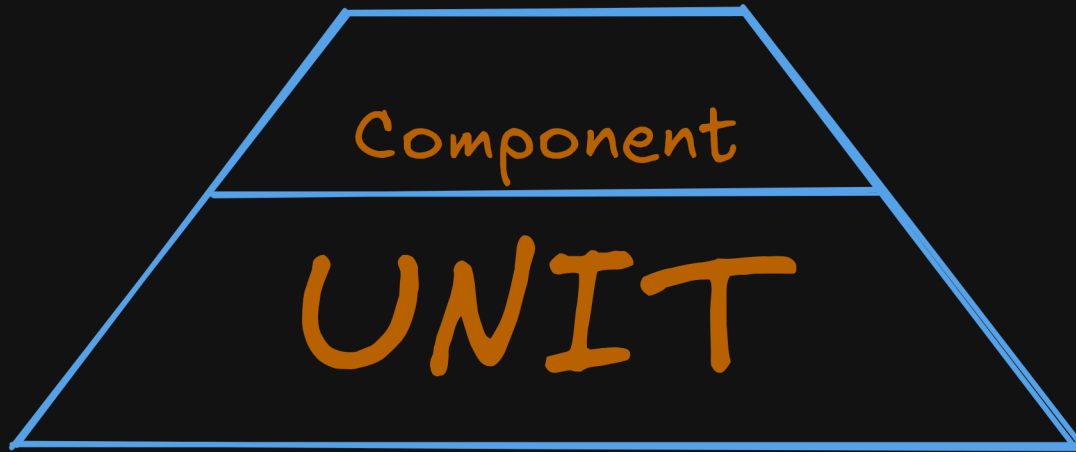
Things we usually forget to test

- empty source/target datasets,
- one scenario per one test

Things we usually forget after unit testing

100% coverage doesn't mean that we're 100% safe

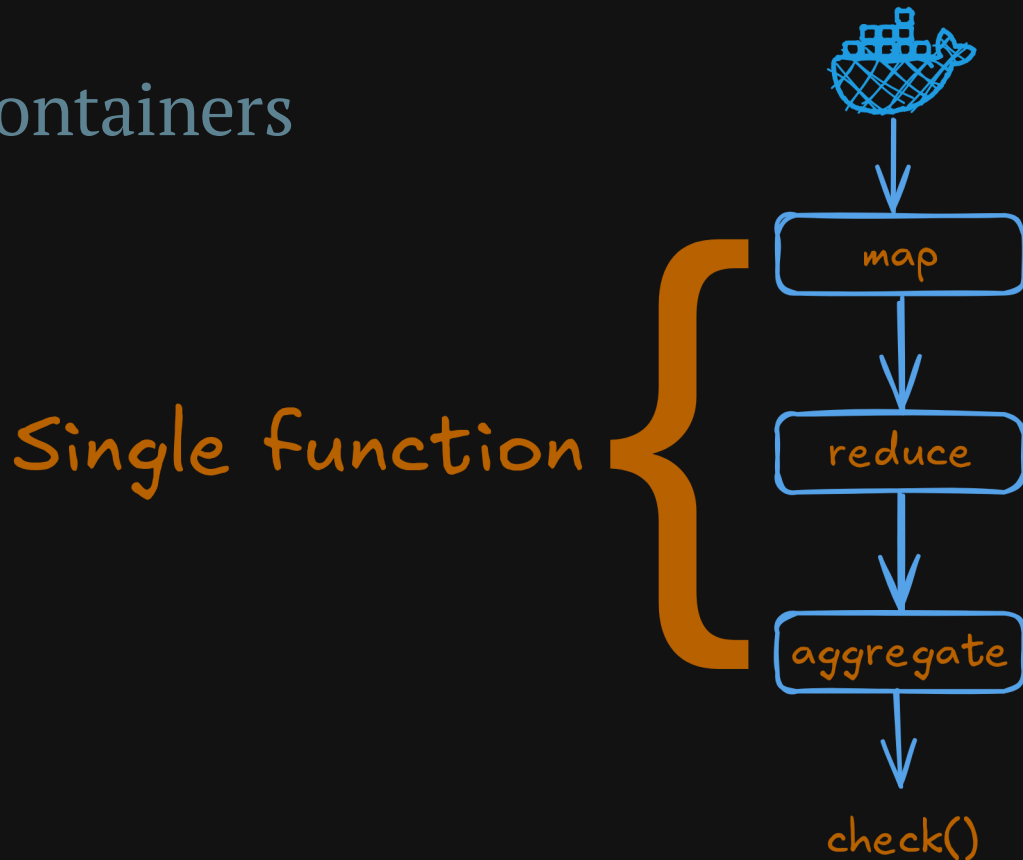
Component testing (or Integration testing)





TESTCONTAINERS

Testcontainers



Testcontainers

Supported languages:

- Java (and compatibles: Scala, Kotlin, etc.)
- LocalStack Module (AWS)
- Python
- Go
- Node.js
- Rust
- .NET

Testcontainers

```
1  @Testcontainers
2  class PostgreSQLIntegrationTest {
3
4      @Container
5      static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:17-alpine")
6          .withDatabaseName("testdb")
7          .withUsername("testuser")
8          .withPassword("testpass");
9
10     @Test
11     void testDatabaseOperations() throws Exception {
12         String jdbcUrl = postgres.getJdbcUrl();
13         // omit here...
14         try (Connection connection = DriverManager.getConnection(jdbcUrl, username, password);
15              Statement statement = connection.createStatement()) {
16             ResultSet resultSet = statement.executeQuery("SELECT email FROM users WHERE");
17             assertTrue(resultSet.next());
18             assertEquals("test@example.com", resultSet.getString("email"));
19         }
```

Testcontainers

```
1  @Testcontainers
2  class PostgreSQLIntegrationTest {
3
4      @Container
5      static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:17-alpine")
6          .withDatabaseName("testdb")
7          .withUsername("testuser")
8          .withPassword("testpass");
9
10     @Test
11     void testDatabaseOperations() throws Exception {
12         String jdbcUrl = postgres.getJdbcUrl();
13         // omit here...
14         try (Connection connection = DriverManager.getConnection(jdbcUrl, username, password);
15              Statement statement = connection.createStatement()) {
16             ResultSet resultSet = statement.executeQuery("SELECT email FROM users WHERE");
17             assertTrue(resultSet.next());
18             assertEquals("test@example.com", resultSet.getString("email"));
19         }
```

Testcontainers

```
1  @Testcontainers
2  class PostgreSQLIntegrationTest {
3
4      @Container
5      static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:17-alpine")
6          .withDatabaseName("testdb")
7          .withUsername("testuser")
8          .withPassword("testpass");
9
10     @Test
11     void testDatabaseOperations() throws Exception {
12         String jdbcUrl = postgres.getJdbcUrl();
13         // omit here...
14         try (Connection connection = DriverManager.getConnection(jdbcUrl, username, password);
15              Statement statement = connection.createStatement()) {
16             ResultSet resultSet = statement.executeQuery("SELECT email FROM users WHERE");
17             assertTrue(resultSet.next());
18             assertEquals("test@example.com", resultSet.getString("email"));
19         }
```

Testcontainers

```
1  @Testcontainers
2  class PostgreSQLIntegrationTest {
3
4      @Container
5      static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:17-alpine")
6          .withDatabaseName("testdb")
7          .withUsername("testuser")
8          .withPassword("testpass");
9
10     @Test
11     void testDatabaseOperations() throws Exception {
12         String jdbcUrl = postgres.getJdbcUrl();
13         // omit here...
14         try (Connection connection = DriverManager.getConnection(jdbcUrl, username, password);
15              Statement statement = connection.createStatement()) {
16             ResultSet resultSet = statement.executeQuery("SELECT email FROM users WHERE");
17             assertTrue(resultSet.next());
18             assertEquals("test@example.com", resultSet.getString("email"));
19         }
```

Testcontainers

```
1  @Testcontainers
2  class PostgreSQLIntegrationTest {
3
4      @Container
5      static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:17-alpine")
6          .withDatabaseName("testdb")
7          .withUsername("testuser")
8          .withPassword("testpass");
9
10     @Test
11     void testDatabaseOperations() throws Exception {
12         String jdbcUrl = postgres.getJdbcUrl();
13         // omit here...
14         try (Connection connection = DriverManager.getConnection(jdbcUrl, username, password)) {
15             Statement statement = connection.createStatement();
16             ResultSet resultSet = statement.executeQuery("SELECT email FROM users WHERE");
17             assertTrue(resultSet.next());
18             assertEquals("test@example.com", resultSet.getString("email"));
19         }
```

Testcontainers

```
1  @Testcontainers
2  class PostgreSQLIntegrationTest {
3
4      @Container
5      static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:17-alpine")
6          .withDatabaseName("testdb")
7          .withUsername("testuser")
8          .withPassword("testpass");
9
10     @Test
11     void testDatabaseOperations() throws Exception {
12         String jdbcUrl = postgres.getJdbcUrl();
13         // omit here...
14         try (Connection connection = DriverManager.getConnection(jdbcUrl, username, password);
15              Statement statement = connection.createStatement()) {
16             ResultSet resultSet = statement.executeQuery("SELECT email FROM users WHERE");
17             assertTrue(resultSet.next());
18             assertEquals("test@example.com", resultSet.getString("email"));
19         }
```


Testcontainers

```
1  @Testcontainers
2  class PostgreSQLIntegrationTest {
3
4      @Container
5      static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:17-alpine")
6          .withDatabaseName("testdb")
7          .withUsername("testuser")
8          .withPassword("testpass");
9
10     @Test
11     void testDatabaseOperations() throws Exception {
12         String jdbcUrl = postgres.getJdbcUrl();
13         // omit here...
14         try (Connection connection = DriverManager.getConnection(jdbcUrl, username, password);
15              Statement statement = connection.createStatement()) {
16             ResultSet resultSet = statement.executeQuery("SELECT email FROM users WHERE");
17             assertTrue(resultSet.next());
18             assertEquals("test@example.com", resultSet.getString("email"));
19         }
```

When Should You Run It?

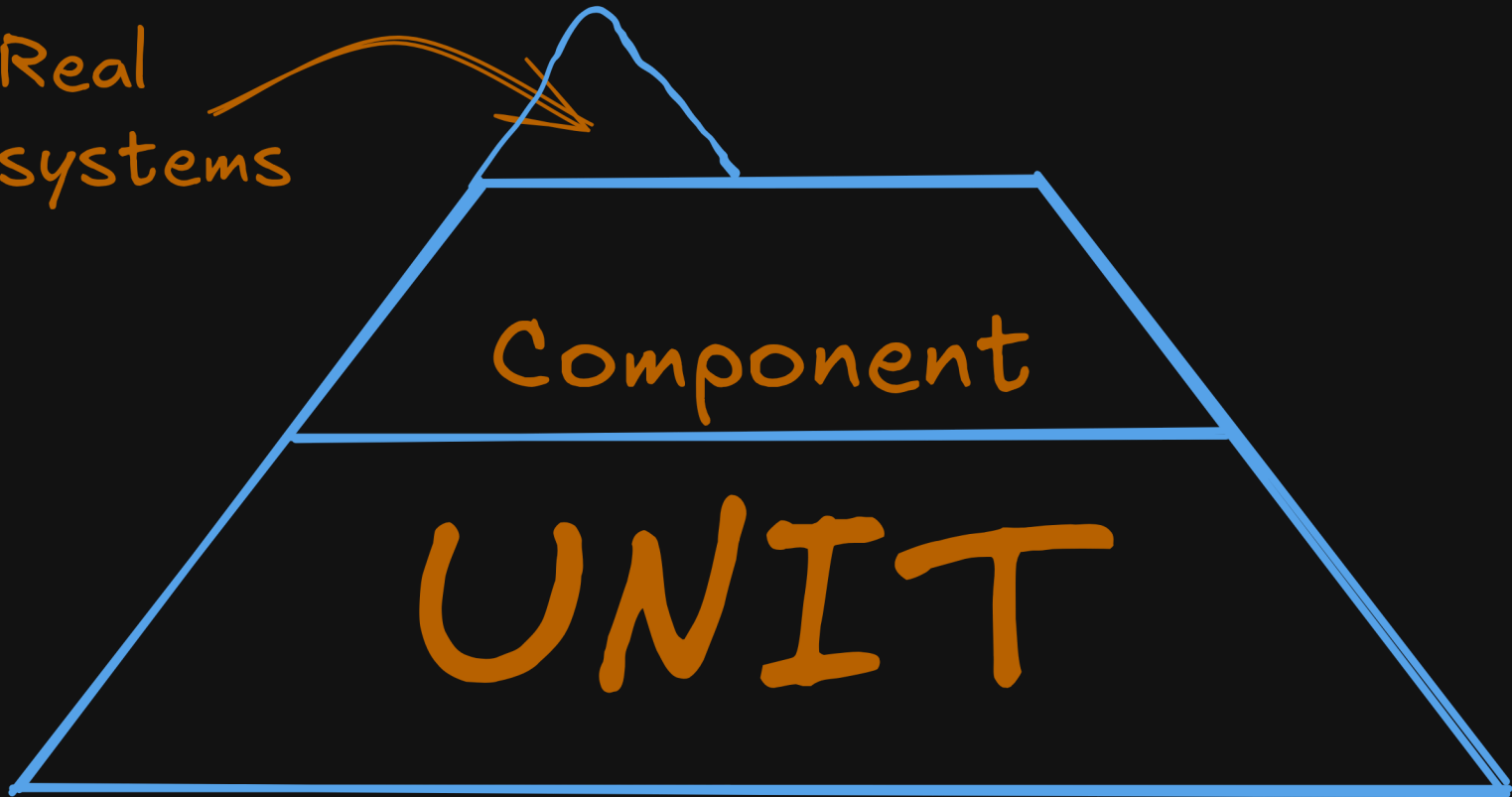
On CI/CD

consider file changes in specific files

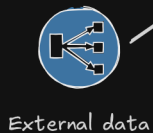
Things we usually forget to test

- reruns (idempotency),
- data duplicates (inside/outside batch),
- incremental runs logic,
- metastore data (partitions, parameters, etc)

Real
systems



Data sources

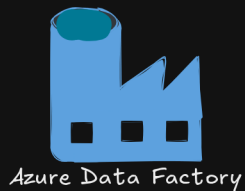


Orchestration

Ingestion



Data storage



Analysis



Visualisation



Real systems

Why are component tests not enough?

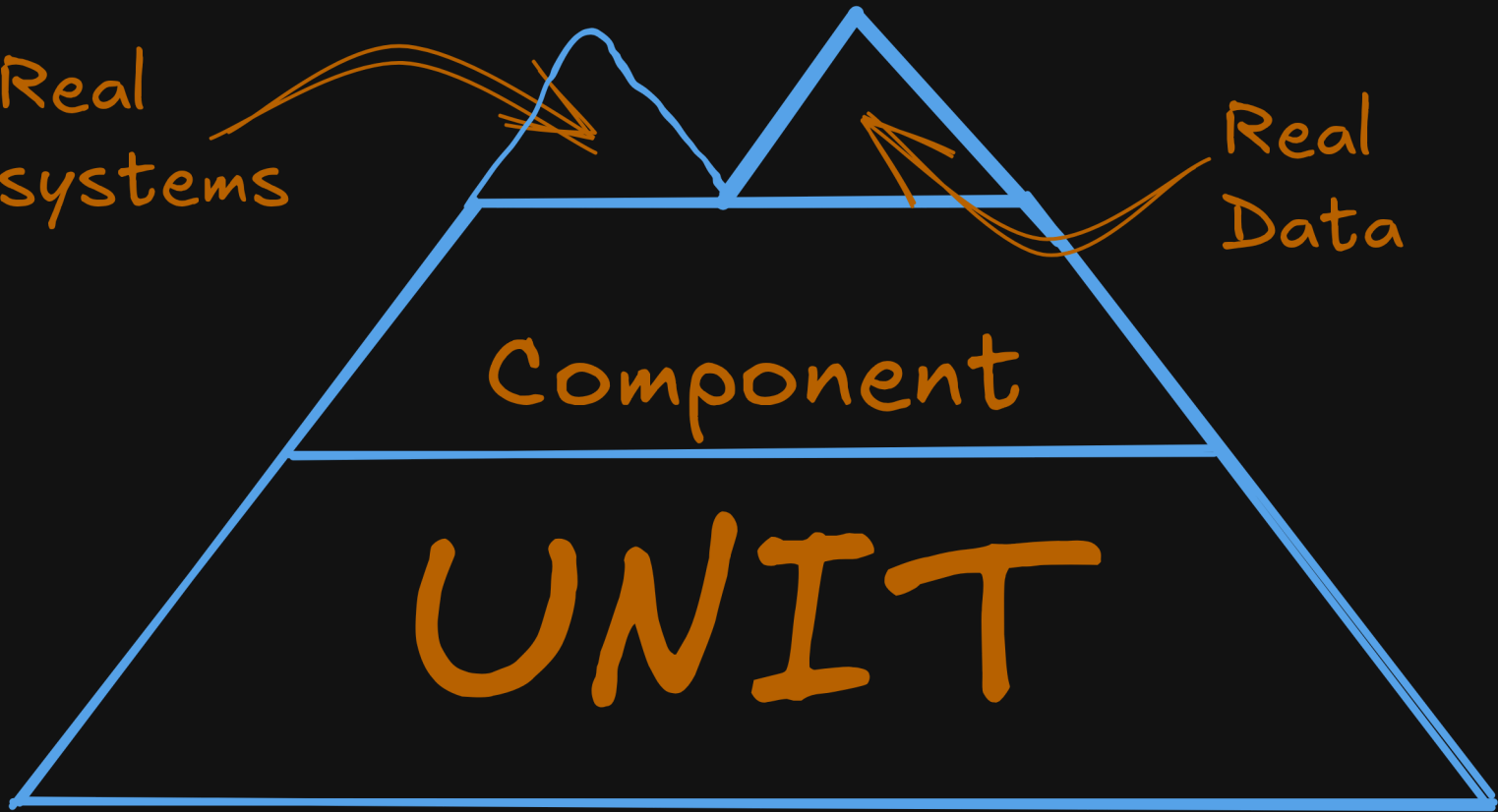
- vendor lock tools (DB, processing, etc.)
- external error handling
- access to data
- overall integration (especially for platform tooling)

Real
systems

Real
Data

Component

UNIT

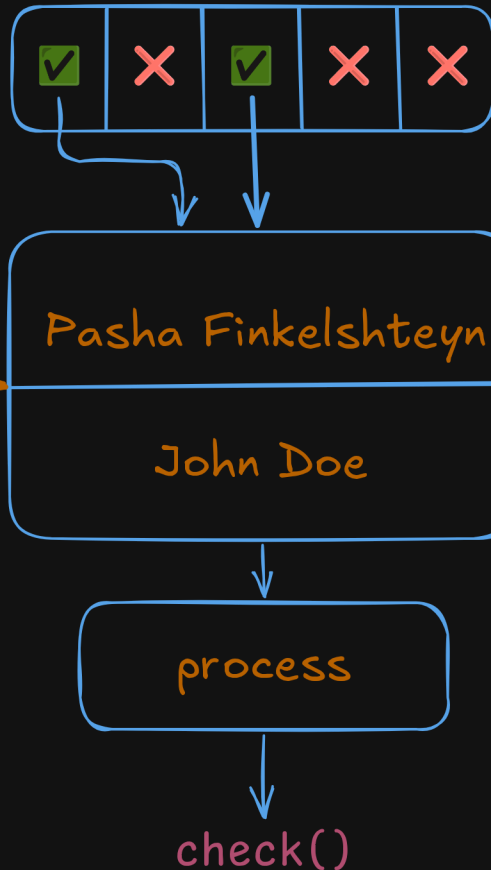


Real data

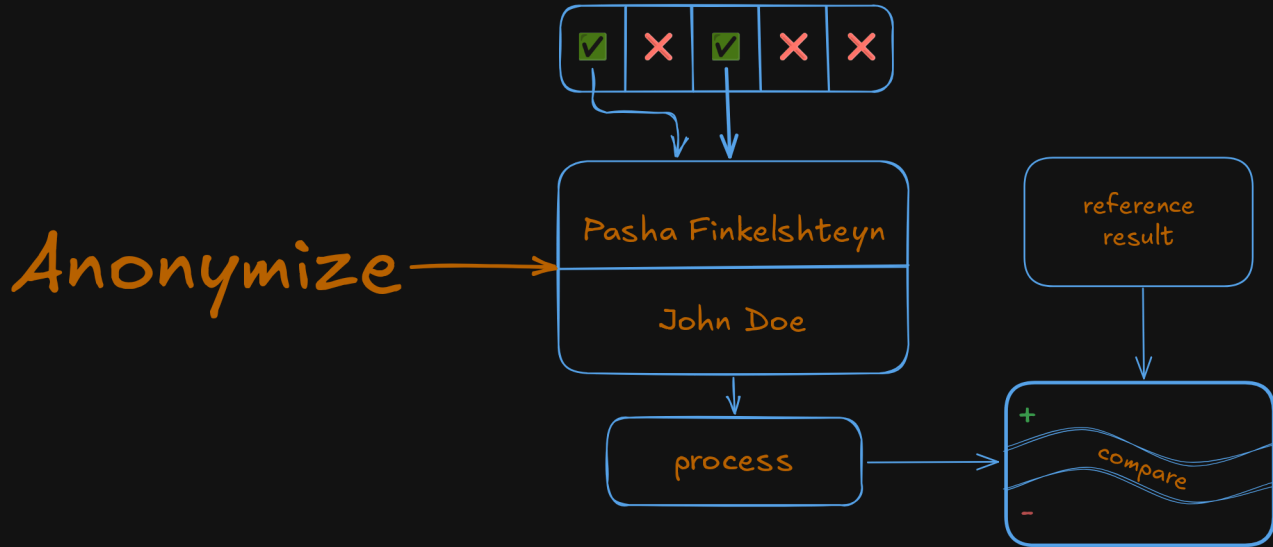
Get data samples from prod

Anonymize it

Anonymize



Compare to reference



Comparison example

gender	reference	id	match
m	m	1	true
f	c	2	false
u	u	3	true
c	c	4	true
m	f	5	false

Real data

Deploy full data backup on stage env, anonymize it 🤖

In usual testing you
won't trust your code

**In pipeline testing you
won't trust neither your
code nor your data**

Real data expectations

Test:

✓ no data

✓ valid data

? invalid data

? illegal data format

Real data expectations. Tools:

- great expectations
- Deequ

Real data expectations

- profilers
- constraint suggestions
- constraint verification
- metrics
- metrics stores


```
1  from pyspark.sql.types import Row, StructType
2  from datetime import datetime
3
4  schema = {
5      "type": "struct",
6      "fields": [
7          {"name": "Id", "type": "long", "nullable": False, metadata: {}},
8          {"name": "SaleDate", "type": "timestamp", "nullable": False, metadata: {}},
9          {"name": "Country", "type": "string", "nullable": False, metadata: {}},
10     ]
11
12     table_rows = [
13         Row(1, datetime(2025, 1, 1, 10, 0, 0), "DE"),
14         Row(2, datetime(1000, 1, 1, 10, 0, 0), "NL"),
15         Row(3, datetime(2023, 1, 1, 10, 0, 0), "NO"),
16         Row(4, datetime(2024, 1, 1, 10, 0, 0), ""),
17     ]
18
19     sample_df = spark.createDataFrame(table_rows, StructType.fromJson(schema))
```



```
1  from pyspark.sql.types import Row, StructType
2  from datetime import datetime
3
4  schema = {
5      "type": "struct",
6      "fields": [
7          {"name": "Id", "type": "long", "nullable": False, metadata: {}},
8          {"name": "SaleDate", "type": "timestamp", "nullable": False, metadata: {}},
9          {"name": "Country", "type": "string", "nullable": False, metadata: {}},
10      ]
11
12  table_rows = [
13      Row(1, datetime(2025, 1, 1, 10, 0, 0), "DE"),
14      Row(2, datetime(1000, 1, 1, 10, 0, 0), "NL"),
15      Row(3, datetime(2023, 1, 1, 10, 0, 0), "NO"),
16      Row(4, datetime(2024, 1, 1, 10, 0, 0), ""),
17  ]
18
19  sample_df = spark.createDataFrame(table_rows, StructType.fromJson(schema))
```



```
1  from pyspark.sql.types import Row, StructType
2  from datetime import datetime
3
4  schema = {
5      "type": "struct",
6      "fields": [
7          {"name": "Id", "type": "long", "nullable": False, metadata: {}},
8          {"name": "SaleDate", "type": "timestamp", "nullable": False, metadata: {}},
9          {"name": "Country", "type": "string", "nullable": False, metadata: {}},
10      ]
11
12  table_rows = [
13      Row(1, datetime(2025, 1, 1, 10, 0, 0), "DE"),
14      Row(2, datetime(1000, 1, 1, 10, 0, 0), "NL"),
15      Row(3, datetime(2023, 1, 1, 10, 0, 0), "NO"),
16      Row(4, datetime(2024, 1, 1, 10, 0, 0), ""),
17  ]
18
19  sample_df = spark.createDataFrame(table_rows, StructType.fromJson(schema))
```



```
1  from pyspark.sql.types import Row, StructType
2  from datetime import datetime
3
4  schema = {
5      "type": "struct",
6      "fields": [
7          {"name": "Id", "type": "long", "nullable": False, metadata: {}},
8          {"name": "SaleDate", "type": "timestamp", "nullable": False, metadata: {}},
9          {"name": "Country", "type": "string", "nullable": False, metadata: {}},
10     ],
11 }
12
13 table_rows = [
14     Row(1, datetime(2025, 1, 1, 10, 0, 0), "DE"),
15     Row(2, datetime(1000, 1, 1, 10, 0, 0), "NL"),
16     Row(3, datetime(2023, 1, 1, 10, 0, 0), "NO"),
17     Row(4, datetime(2024, 1, 1, 10, 0, 0), ""),
18 ]
19
20 sample_df = spark.createDataFrame(table_rows, StructType.fromJson(schema))
```



Great Expectations

```
1  {
2    "result": {
3      "element_count": 4,
4      "unexpected_count": 2,
5      "unexpected_percent": 50.0,
6      "partial_unexpected_list": ["NO", ""]
7    },
8    "success": false,
9    "expectation_config": {
10     "kwargs": {
11       "column": "Country",
12       "value_set": ["DE", "NL"]
13     }
14   }
15 }
```



Great Expectations

```
1  {
2    "result": {
3      "element_count": 4,
4      "unexpected_count": 2,
5      "unexpected_percent": 50.0,
6      "partial_unexpected_list": ["NO", ""]
7    },
8    "success": false,
9    "expectation_config": {
10     "kwargs": {
11       "column": "Country",
12       "value_set": ["DE", "NL"]
13     }
14   }
15 }
```



Great Expectations

```
1  {
2    "result": {
3      "element_count": 4,
4      "unexpected_count": 2,
5      "unexpected_percent": 50.0,
6      "partial_unexpected_list": ["NO", ""]
7    },
8    "success": false,
9    "expectation_config": {
10     "kwargs": {
11       "column": "Country",
12       "value_set": ["DE", "NL"]
13     }
14   }
15 }
```



Great Expectations

```
1  {
2    "result": {
3      "element_count": 4,
4      "unexpected_count": 2,
5      "unexpected_percent": 50.0,
6      "partial_unexpected_list": ["NO", ""]
7    },
8    "success": false,
9    "expectation_config": {
10     "kwargs": {
11       "column": "Country",
12       "value_set": ["DE", "NL"]
13     }
14   }
15 }
```



Java Deequ

```
1 SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();
2
3 Dataset<Row> data = spark.sqlContext().read().parquet("/some/path/dataset");
4
5 Check check = new Check(spark, CheckLevel.Error(), "Review Check");
6
7 VerificationSuite checkResult = new VerificationSuite(spark)
8     .onData(data)
9     .addCheck(check
10         .isComplete("Country")
11         .isContainedIn("Country", List.of("DE", "NL")))
12     .run();
13
14 verificationDf = VerificationResult.checkResultsAsDataFrame(spark, checkResult);
15 verificationDf.show()
```



Java Deequ

```
1  SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();
2
3  Dataset<Row> data = spark.sqlContext().read().parquet("/some/path/dataset");
4
5  Check check = new Check(spark, CheckLevel.Error(), "Review Check");
6
7  VerificationSuite checkResult = new VerificationSuite(spark)
8      .onData(data)
9      .addCheck(check
10         .isComplete("Country")
11         .isContainedIn("Country", List.of("DE", "NL")))
12      .run();
13
14  verificationDf = VerificationResult.checkResultsAsDataFrame(spark, checkResult);
15  verificationDf.show()
```



Java Deequ

```
1 SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();
2
3 Dataset<Row> data = spark.sqlContext().read().parquet("/some/path/dataset");
4
5 Check check = new Check(spark, CheckLevel.Error(), "Review Check");
6
7 VerificationSuite checkResult = new VerificationSuite(spark)
8     .onData(data)
9     .addCheck(check
10         .isComplete("Country")
11         .isContainedIn("Country", List.of("DE", "NL")))
12     .run();
13
14 verificationDf = VerificationResult.checkResultsAsDataFrame(spark, checkResult);
15 verificationDf.show()
```



Java Deequ

```
1 SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();
2
3 Dataset<Row> data = spark.sqlContext().read().parquet("/some/path/dataset");
4
5 Check check = new Check(spark, CheckLevel.Error(), "Review Check");
6
7 VerificationSuite checkResult = new VerificationSuite(spark)
8     .onData(data)
9     .addCheck(check
10         .isComplete("Country")
11         .isContainedIn("Country", List.of("DE", "NL")))
12     .run();
13
14 verificationDf = VerificationResult.checkResultsAsDataFrame(spark, checkResult);
15 verificationDf.show()
```



Java Deequ

```
1 SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();
2
3 Dataset<Row> data = spark.sqlContext().read().parquet("/some/path/dataset");
4
5 Check check = new Check(spark, CheckLevel.Error(), "Review Check");
6
7 VerificationSuite checkResult = new VerificationSuite(spark)
8     .onData(data)
9     .addCheck(check
10         .isComplete("Country")
11         .isContainedIn("Country", List.of("DE", "NL")))
12     .run();
13
14 verificationDf = VerificationResult.checkResultsAsDataFrame(spark, checkResult);
15 verificationDf.show()
```



Java Deequ

```
1 SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();
2
3 Dataset<Row> data = spark.sqlContext().read().parquet("/some/path/dataset");
4
5 Check check = new Check(spark, CheckLevel.Error(), "Review Check");
6
7 VerificationSuite checkResult = new VerificationSuite(spark)
8     .onData(data)
9     .addCheck(check
10         .isComplete("Country")
11         .isContainedIn("Country", List.of("DE", "NL")))
12     .run();
13
14 verificationDf = VerificationResult.checkResultsAsDataFrame(spark, checkResult);
15 verificationDf.show()
```



Java Deequ

```
1 SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();
2
3 Dataset<Row> data = spark.sqlContext().read().parquet("/some/path/dataset");
4
5 Check check = new Check(spark, CheckLevel.Error(), "Review Check");
6
7 VerificationSuite checkResult = new VerificationSuite(spark)
8     .onData(data)
9     .addCheck(check
10         .isComplete("Country")
11         .isContainedIn("Country", List.of("DE", "NL")))
12     .run();
13
14 verificationDf = VerificationResult.checkResultsAsDataFrame(spark, checkResult);
15 verificationDf.show()
```



Java Deequ

```
1 SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();
2
3 Dataset<Row> data = spark.sqlContext().read().parquet("/some/path/dataset");
4
5 Check check = new Check(spark, CheckLevel.Error(), "Review Check");
6
7 VerificationSuite checkResult = new VerificationSuite(spark)
8     .onData(data)
9     .addCheck(check
10         .isComplete("Country")
11         .isContainedIn("Country", List.of("DE", "NL")))
12     .run();
13
14 verificationDf = VerificationResult.checkResultsAsDataFrame(spark, checkResult);
15 verificationDf.show()
```



Java Deequ

```
1 SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();
2
3 Dataset<Row> data = spark.sqlContext().read().parquet("/some/path/dataset");
4
5 Check check = new Check(spark, CheckLevel.Error(), "Review Check");
6
7 VerificationSuite checkResult = new VerificationSuite(spark)
8     .onData(data)
9     .addCheck(check
10         .isComplete("Country")
11         .isContainedIn("Country", List.of("DE", "NL")))
12     .run();
13
14 verificationDf = VerificationResult.checkResultsAsDataFrame(spark, checkResult);
15 verificationDf.show()
```



Java Deequ. Results

constraint	constraint_status	constraint_message
CompletenessConstraint	Success	
ComplianceConstraint	Failure	Value: 0.5 does not meet the constraint requirement!

Real data expectations. Use cases

Real data expectations. Use cases

- pre-ingestion and post-ingestion data validation

Real data expectations. Use cases

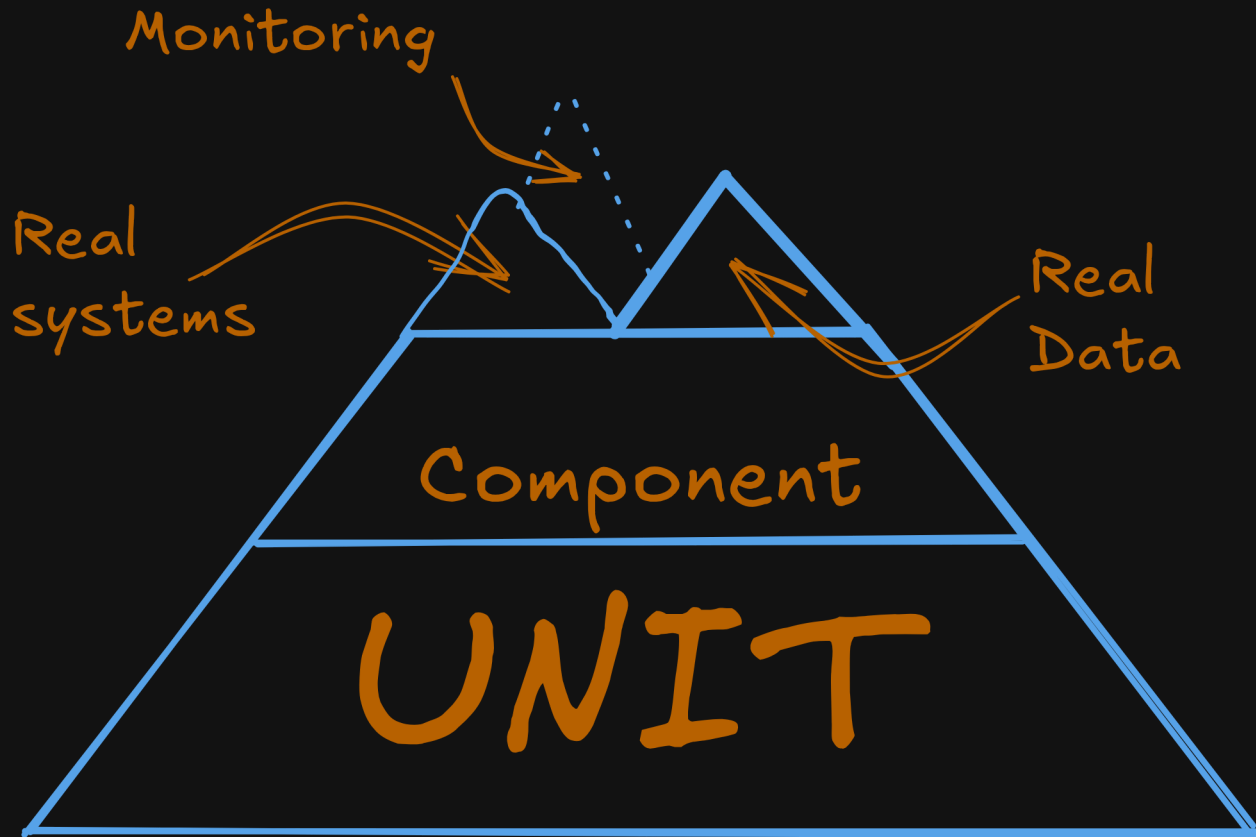
- pre-ingestion and post-ingestion data validation
- before pipeline development

Real data expectations. Use cases

- pre-ingestion and post-ingestion data validation
- before pipeline development
- monitoring and alerting

When Should You Run It?

In DAG run/on schedule



Monitoring

Why?

- The only REAL testing is production
- Data tends to change over time

Monitoring

What?

Monitoring

What?

- data volumes

Monitoring

What?

- data volumes
- time (SLAs)

Monitoring

What?

- data volumes
- time (SLAs)
- dead letter queue monitoring

Monitoring

What?

- data volumes
- time (SLAs)
- dead letter queue monitoring
- service health

Monitoring

What?

- data volumes
- time (SLAs)
- dead letter queue monitoring
- service health
- business metrics

Monitoring

How?

Monitoring

How?

- use Listeners

Monitoring

How?

- use Listeners
- Data Aggregators

Monitoring

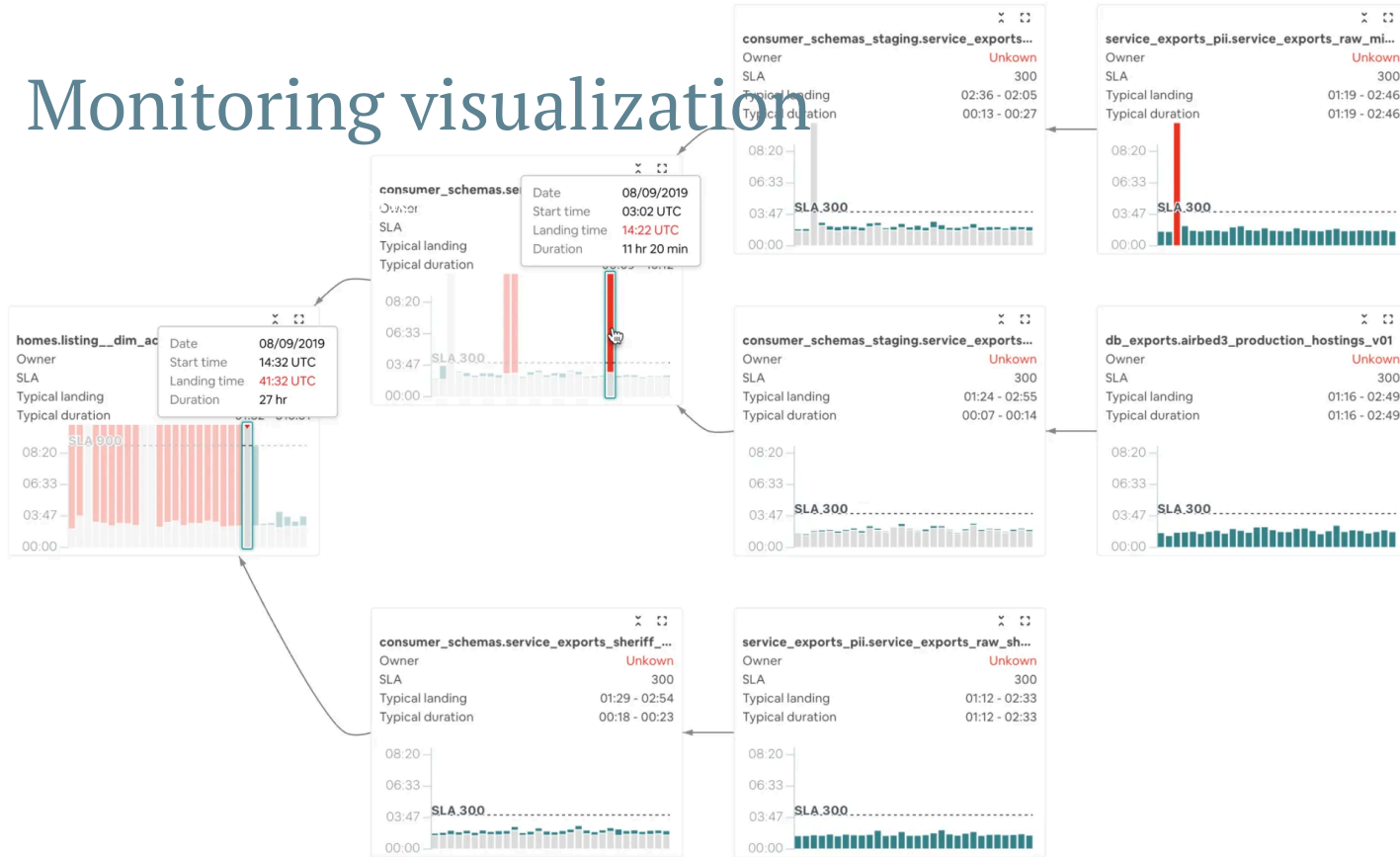
How?

- use Listeners
- Data Aggregators
- Airflow (Dagster, Prefect, etc)

Data pipeline is always a DAG

Monitoring should visualize it

Monitoring visualization

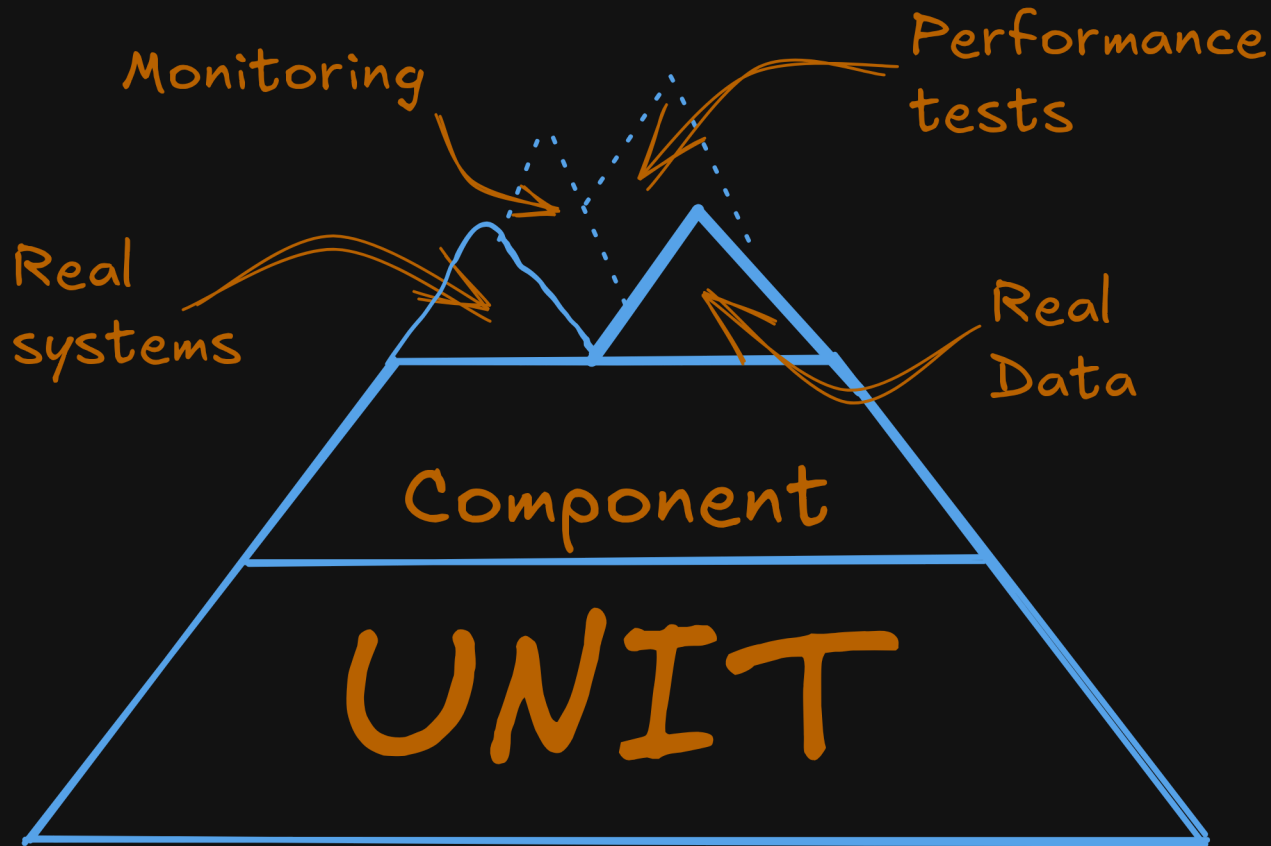


End-to-End tests

Compare with reports, old DWH

Multiple dimensions:

- data
- data latency
- performance, scalability



Performance Tests

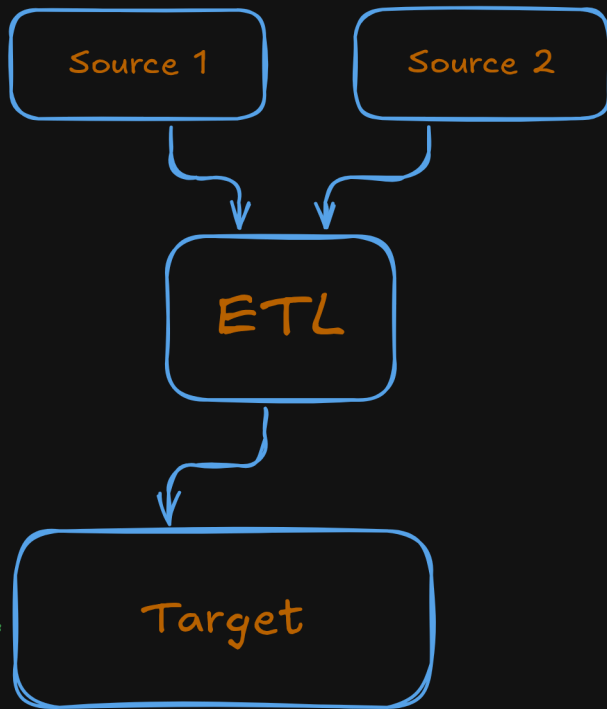
- start with SLO
- test your initial data load

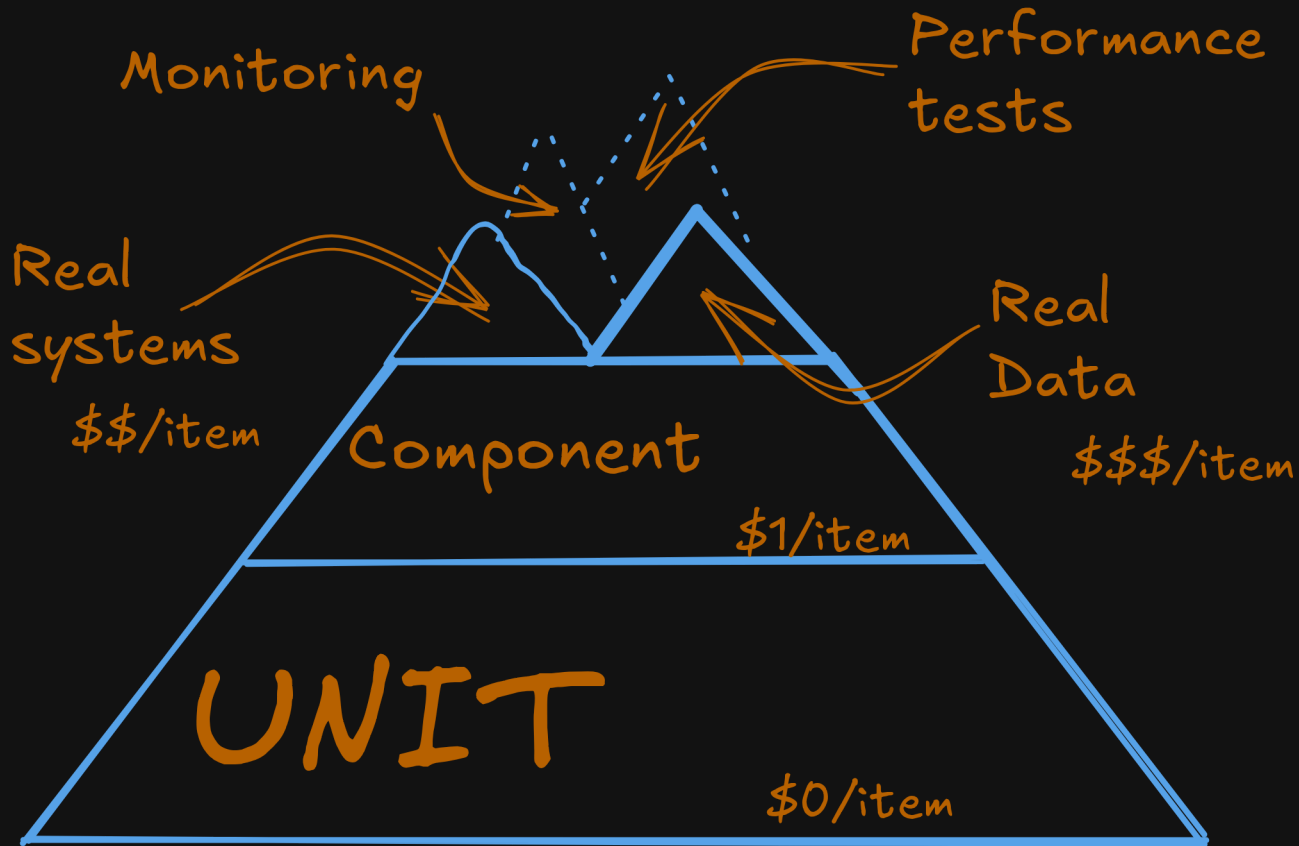
Real prod

Run a parallel job with a different sink



check()





Summary

Summary

- Testing pipeline is like testing code

Summary

- Testing pipeline is like testing code
- Testing pipelines is not like testing code

Summary

- Testing pipeline is like testing code
- Testing pipelines is not like testing code
- Pipeline quality is not only about testing

Summary

- Testing pipeline is like testing code
- Testing pipelines is not like testing code
- Pipeline quality is not only about testing
- Sometimes testing outside of production is tricky

Thanks!

Questions? 

@asm0di0

@if_no_then_yes

END